ParameterManager.cc - Original Implementation Analysis

Overview

This document explains how the original ParameterManager.cc works before the robustness improvements, function by function and variable by variable.

Key Classes and Data Structures

Main Class: ParameterManager

The ParameterManager is responsible for loading, storing, and managing all vehicle parameters (settings) from the autopilot.

Core Variables and Their Purpose

1. Connection and State Variables

```
Vehicle* _vehicle;                    // Reference to the vehicle object
MAVLinkProtocol* _mavlink;           // MAVLink communication protocol handler
bool _parametersReady;               // True when all parameters are loaded and ready
bool _missingParameters;             // True if some parameters failed to load
bool _initialLoadComplete;           // True when initial parameter loading is done
bool _waitingForDefaultComponent;    // True if waiting for default component params
```

2. Progress Tracking

```
double _loadProgress;                // Loading progress from 0.0 to 1.0
int _totalParamCount;               // Total number of parameters across all components
QMap<int, int> _paramCountMap;      // Maps component ID to expected parameter count
```

3. Parameter Storage

```
QMap<int, QMap<QString, Fact*>> _mapCompId2FactMap;
// Structure: Component ID -> Parameter Name -> Fact Object
// This is the main storage for all loaded parameters
```

4. Waiting Lists (Critical for Understanding)

```
QMap<int, QMap<int, int>> _waitingReadParamIndexMap;
// Structure: Component ID -> Parameter Index -> Retry Count
// Tracks which parameter indices we're still waiting for

QMap<int, QMap<QString, int>> _waitingReadParamNameMap;
// Structure: Component ID -> Parameter Name -> Retry Count
// Tracks which parameter names we're still waiting for

QMap<int, QList<int>> _failedReadParamIndexMap;
// Structure: Component ID -> List of Failed Indices
// Stores indices that failed after max retries
```

5. Retry and Timeout Control

```
QTimer _initialRequestTimeoutTimer;  // Timer for initial parameter request
QTimer _waitingParamTimeoutTimer;    // Timer for waiting for parameters
int _initialRequestRetryCount;       // How many times we've retried initial request
static const int _maxInitialRequestListRetry = 4;        // Max retries for initial request
```

```
static const int _maxInitialLoadRetrySingleParam = 5;    // Max retries per parameter
```

6. FTP Support

```
bool _tryftp;                       // True if we should try FTP parameter download first
```

Core Functions and Their Behavior

1. Constructor: ParameterManager(Vehicle* vehicle)

Purpose: Initialize the parameter manager What it does: - Sets up timers with 5-second and 3-second intervals - Connects timer signals to timeout handlers - Creates parameter cache directory - Sets initial state variables to false/0

2. refreshAllParameters(uint8_t componentId)

Purpose: Start the parameter loading process What it does: - Checks if link is high-latency (skips parameter loading if so) - For ArduPilot: Tries FTP download first - For others: Sends PARAM_REQUEST_LIST message - Resets waiting lists for index-based requests Critical Behavior: - If FTP fails, falls back to normal MAVLink requests - Sets up waiting lists based on known parameter counts

3. mavlinkMessageReceived(mavlink_message_t message)

Purpose: Handle incoming MAVLink messages What it does: - Filters for PARAM_VALUE messages - Decodes parameter data (name, value, type, count, index) - Calls _handleParamValue with decoded data Critical Behavior: - Only processes PARAM_VALUE messages - Ignores other message types

Critical Issues in Original Implementation

1. Early Parameter Loss: Parameters with index 65535 are discarded. Impact: Can cause 'missing parameters' errors. Example: STAT_RUNTIME parameter often arrives early and gets lost.
2. All-or-Nothing Approach: If ANY parameter fails, entire system marked as 'missing parameters'. Impact: UI may not work even if 99% of parameters loaded successfully.
3. Limited Retry Strategies: Only retries by index, no name-based fallback. Impact: Some parameters may be recoverable by name-based requests.
4. No Coverage Analysis: System fails even with high parameter coverage. Impact: 995/1000 parameters loaded (99.5%) still causes failure.
5. Rigid Timeout Handling: Fixed timeouts regardless of connection quality. Impact: Poor connections cause unnecessary failures.

Summary

The original ParameterManager was designed with a 'perfect or nothing' philosophy. While conservative from a safety perspective, it can be overly rigid and cause failures in real-world conditions with imperfect connections or firmware quirks.